



GRUPO 10 · REQUERIMIENTOS FUNCIONALES

Administración de Roles y Permisos

Módulo: Gestión de usuarios internos (backoffice)

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 10
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aikuaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

Documento derivado del código fuente de aikuaa-sandbox-api y aikuaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el catálogo de roles del backoffice y la asignación y revocación de roles a un usuario. La revocación es una baja lógica: la asignación permanece registrada y se reactiva si el rol vuelve a otorgarse.

Fuera de alcance: Definición de permisos concretos por rol, verificación de permisos al ejecutar operaciones, alta y baja de roles del catálogo, jerarquías y aprobación de asignaciones por un superior.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de usuarios)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos qa_api (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy /api/proxy/{path}; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/roles	RF-G10-01	Devuelve el catálogo completo de roles.
GET /api/v1/usuarios/{id}/roles	RF-G10-02	Lista los roles vigentes de un usuario.
POST /api/v1/usuarios/{id}/roles	RF-G10-03	Asigna un rol a un usuario, o reactiva una asignación revocada.
DELETE /api/v1/usuarios/{id}/roles/{roleId}	RF-G10-04	Revoca un rol de un usuario mediante baja lógica.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key."
- Cada API key admite un máximo de **30 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE`, y las operaciones que "eliminan" hacen baja lógica (`activo = false`).

- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- El catálogo de roles es fijo y se carga con los datos sembrados: `admin`, `soporte`, `auditor` y `operador`. La API no permite crear ni eliminar roles.
- Un mismo usuario no puede tener dos asignaciones del mismo rol: la base lo impide con una restricción de unicidad, y la asignación la contempla reactivando la fila existente.

3. Modelo de datos del módulo

roles

Catálogo de roles del backoffice. El módulo solo lo lee.

Columna	Tipo / restricción
id	bigserial, clave primaria
nombre	text, único, uno de admin / soporte / auditor / operador
descripcion	text, opcional
created_at	timestampz, por defecto now()

usuario_roles

Asignaciones de rol a usuario. Es la tabla que escribe el módulo.

Columna	Tipo / restricción
id	bigserial, clave primaria
usuario_id	bigint, obligatorio, referencia a usuarios(id)
role_id	bigint, obligatorio, referencia a roles(id)
activo	boolean, obligatorio, por defecto true; es lo que distingue vigente de revocada
asignado_en	timestampz, por defecto now(); no se actualiza al reactivar
Restricción	La combinación de usuario y rol es única: una sola fila por par

El recurso `roles` de este módulo (permisos de backoffice) no tiene relación con el grupo de curso del alumno, que se consulta por separado con `GET /api/v1/roster?email=`. Son dos conceptos distintos que comparten un nombre parecido.

4. Requerimientos funcionales

RF-G10-01 — Consultar el catálogo de roles

GET /api/v1/roles

Devuelve todos los roles disponibles para asignar, con su nombre y su descripción. Es también el endpoint más liviano de la API, por lo que la aplicación web lo usa para comprobar que una API key es válida.

Entradas y validaciones

- No recibe parámetros.

Reglas de negocio

- Devuelve el catálogo completo ordenado por identificador ascendente, sin filtros ni paginación.
- El catálogo es fijo: la API no permite crear, modificar ni eliminar roles.
- Los identificadores de rol que devuelve este endpoint son los que se usan para asignar y revocar.

Respuesta esperada

- 200 OK con data como arreglo de roles, cada uno con id, nombre, descripcion y created_at.

Errores

Código	HTTP	Cuándo ocurre
UNAUTHORIZED	401	Falta la API key o es inválida.

Fuente: app/api/v1/roles/route.ts

RF-G10-02 — Listar los roles vigentes de un usuario

GET /api/v1/usuarios/{id}/roles

Devuelve los roles actualmente vigentes de un usuario, con el nombre y la descripción de cada rol.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo del usuario.

Reglas de negocio

- Devuelve únicamente las asignaciones vigentes (`activo = true`): las revocadas quedan fuera del resultado.
- Cada elemento combina los datos de la asignación con el nombre y la descripción del rol.
- El resultado se ordena por identificador de asignación ascendente.
- Un usuario sin roles vigentes, y también un usuario inexistente, devuelven una lista vacía con estado 200: **este endpoint no distingue ambos casos**.

Respuesta esperada

- 200 OK con `data` como arreglo; cada elemento incluye `id`, `usuario_id`, `role_id`, `activo`, `asignado_en`, `nombre` y `descripcion`.

Errores

Código	HTTP	Cuándo ocurre
VALIDATION_ERROR	400	El identificador de usuario no es un entero positivo.
UNAUTHORIZED	401	Falta la API key o es inválida.

Fuente: `app/api/v1/usuarios/[id]/roles/route.ts`

RF-G10-03 — Asignar un rol a un usuario

POST /api/v1/usuarios/{id}/roles

Otorga un rol del catálogo a un usuario. Si ese rol ya le había sido otorgado y luego revocado, la asignación original se reactiva en lugar de crearse una nueva.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo del usuario.
- `roleId` (obligatorio, en el cuerpo): número entero positivo del rol.

Reglas de negocio

- Si el usuario no tenía ese rol, se crea la asignación con `activo = true`.
- Si ya existía una asignación de ese rol (vigente o revocada), la misma fila se marca como vigente: no se duplica y conserva su identificador original.
- Al reactivar, la fecha de asignación original **no** se actualiza: sigue reflejando el primer otorgamiento.
- Volver a asignar un rol ya vigente es una operación aceptada, sin efecto observable.
- El usuario y el rol deben existir: si alguno no existe, la operación falla como error de ejecución por violación de clave foránea.
- **No se verifica quién asigna:** cualquier portador de una API key válida puede otorgar el rol `admin` a cualquier usuario.

Respuesta esperada

- `201 Created` con `data` conteniendo la asignación resultante (`id`, `usuario_id`, `role_id`, `activo`, `asignado_en`). El estado es 201 tanto en el alta como en la reactivación.

Errores

Código	HTTP	Cuándo ocurre
VALIDATION_ERROR	400	Falta <code>roleId</code> , o alguno de los identificadores no es un entero positivo.
EXECUTION_ERROR	400	El usuario o el rol indicados no existen.

Fuente: `app/api/v1/usuarios/[id]/roles/route.ts`

RF-G10-04 — Revocar un rol de un usuario

DELETE /api/v1/usuarios/{id}/roles/{roleId}

Quita un rol a un usuario. La asignación no se borra: se marca como no vigente, de modo que quede registro de que existió.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo del usuario.
- `roleId` (obligatorio, en la ruta): número entero positivo del rol.

Reglas de negocio

- La revocación es una baja lógica: la fila permanece con `activo = false`. El sistema nunca borra físicamente la asignación.
- Solo se revoca la asignación que corresponde exactamente a ese usuario y ese rol.
- Si el par usuario-rol nunca existió, la operación responde 404 con el mensaje "Asignación de rol no encontrada".
- Revocar una asignación ya revocada devuelve 200: la fila existe, y se vuelve a marcar como no vigente.
- La revocación no elimina ningún dato del usuario ni afecta sus otros roles.

Respuesta esperada

- `200 OK` con `data` conteniendo la asignación con `activo` en falso.

Errores

Código	HTTP	Cuándo ocurre
NOT_FOUND	404	No existe una asignación de ese rol para ese usuario (mensaje: "Asignación de rol no encontrada").
VALIDATION_ERROR	400	Alguno de los identificadores no es un entero positivo.

Fuente: `app/api/v1/usuarios/[id]/roles/[roleId]/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body".

- **Errores de base de datos:** una violación de restricción (CHECK, UNIQUE, clave foránea) no aborta el servicio: se traduce a `400 EXECUTION_ERROR` con el mensaje que devuelve PostgreSQL.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear` , `activar` , `confirmar` , `cancelar` , `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.

Códigos de error

Código	HTTP	Significado
UNAUTHORIZED	401	Falta el header <code>x-api-key</code> , o la clave no existe o está inactiva.
RATE_LIMITED	429	Se superaron las 30 requests por minuto de esa API key.
VALIDATION_ERROR	400	La entrada no cumple el esquema de la ruta, o el JSON es inválido.
EXECUTION_ERROR	400	La consulta falló en la base de datos (restricción violada, tipo inválido).
NOT_FOUND	404	El recurso indicado no existe, o no está en un estado que admita la operación.
INTERNAL_ERROR	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}` , que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local.

Pantalla	Qué permite hacer
<code>/roles</code> — Roles del usuario	Tabla con el catálogo de roles y, para cada uno, si está vigente para el usuario en sesión, con las acciones de asignar y revocar. Ejecuta los cuatro requerimientos del módulo.

- La pantalla opera siempre sobre el usuario que está en sesión: no hay selector de usuario, por lo que probar la asignación a otro usuario requiere llamar la API directamente.
- Después de asignar o revocar, la pantalla vuelve a consultar el estado y refleja el cambio sin recargar.
- El catálogo de roles es además la comprobación que hace la pantalla de acceso para validar una API key.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G10-01 — Consultar el catálogo de roles

- Al consultar el catálogo con una API key válida, la respuesta es 200 y `data` contiene los cuatro roles del sandbox.
- Los nombres devueltos pertenecen al conjunto `admin`, `soporte`, `auditor` y `operador`.
- Al consultar el catálogo sin API key, la respuesta es 401 `UNAUTHORIZED`; este es el mismo control que usa la pantalla de acceso de la web para validar la clave.
- Consultas sucesivas devuelven el mismo catálogo: no hay endpoint que lo modifique.

RF-G10-02 — Listar los roles vigentes de un usuario

- Dado un usuario con un rol asignado, al listar sus roles la respuesta es 200 y el rol figura con `activo` en verdadero y su `nombre` resuelto.
- Tras revocar ese rol, el listado deja de incluirlo.
- Al listar los roles de un usuario inexistente, la respuesta es 200 con `data` vacío, no un 404.
- Al listar con un identificador no numérico, la respuesta es 400 `VALIDATION_ERROR`.

RF-G10-03 — Asignar un rol a un usuario

- Al asignar un rol nuevo a un usuario, la respuesta es 201, `data.activo` es verdadero y el rol aparece en el listado de roles vigentes.
- Al asignar dos veces el mismo rol al mismo usuario, la segunda respuesta también es 201 y el listado sigue mostrando una única asignación de ese rol.
- Al revocar un rol y volver a asignarlo, la asignación conserva el mismo `id` que tenía antes de la revocación.
- Tras esa reactivación, `asignado_en` conserva la fecha del primer otorgamiento.
- Al asignar un `roleId` inexistente, la respuesta es 400 `EXECUTION_ERROR` y no se crea ninguna asignación.
- Al asignar un rol a un usuario inexistente, la respuesta es 400 `EXECUTION_ERROR`.

RF-G10-04 — Revocar un rol de un usuario

- Dado un usuario con un rol vigente, al revocarlo la respuesta es 200 y `data.activo` es falso.
- Tras la revocación, el rol ya no aparece en el listado de roles vigentes del usuario.
- Al revocar dos veces el mismo rol, la segunda respuesta también es 200: la asignación sigue existiendo como fila.
- Al revocar un rol que el usuario nunca tuvo, la respuesta es 404 `NOT_FOUND` con el mensaje "Asignación de rol no encontrada".
- Revocar un rol no afecta a los demás roles vigentes del usuario.

8. Anexo normativo (BCP / SEDECO) y brechas

Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Marco del BCP sobre control interno y seguridad de la información en entidades financieras	Los accesos privilegiados deben otorgarse bajo aprobación, con segregación de funciones y revisión periódica.	No implementado: cualquier portador de una API key válida puede otorgarse o quitar cualquier rol, sin aprobación ni segregación.
Principio de mínimo privilegio	Cada usuario debe tener únicamente los permisos necesarios para su función.	No implementado: los roles son etiquetas; ninguna operación del sandbox verifica el rol del usuario antes de ejecutarse.
Trazabilidad de altas y bajas de accesos	Debe quedar registro de quién otorgó o revocó cada permiso y cuándo.	Parcial: la baja lógica conserva la asignación y toda request queda auditada, pero la asignación no guarda quién la ejecutó, y la fecha no se actualiza al reactivarla.
Ley 6534/2020 de protección de datos personales	El acceso a datos personales debe estar restringido según el rol del funcionario.	No implementado: todos los endpoints exponen los mismos datos a cualquier API key, con independencia de los roles asignados.
Revisión periódica de accesos (recertificación)	Los permisos vigentes deben revisarse periódicamente y revocarse los innecesarios.	Parcial: el listado de roles vigentes permite la revisión, pero no hay vencimiento ni proceso de recertificación.

Brechas conocidas del sandbox

- Los roles no otorgan ni restringen nada: ningún endpoint verifica el rol del usuario antes de operar.
- Cualquier API key válida puede asignar el rol `admin` a cualquier usuario, incluido a sí mismo.
- El listado de roles de un usuario inexistente devuelve una lista vacía con estado 200, en lugar de 404.
- La asignación no registra quién la realizó, y al reactivar una asignación revocada no se actualiza la fecha de otorgamiento.
- El catálogo de roles es fijo: no hay alta, baja ni edición de roles, ni un modelo de permisos asociado.
- No hay vencimiento de asignaciones ni proceso de recertificación de accesos.